

基于瑞芯微 RK3588 的甘蔗地智慧农业云端作业系统

摘要

南方甘蔗生长中易受草害影响，传统除草方式存在效率低、杂草位置难以数字化管理、粗放式除草剂喷洒易造成药液大量浪费和农田破坏等问题。针对以上痛点，作品设计并实现了一套基于瑞芯微 RK3588 的甘蔗地数字化建图与靶向喷药作业系统。系统采用 RK3588+STM32 双处理器架构：RK3588 运行 ROS2，利用 NPU 部署 YOLOv8/RKNN 模型实现作物与杂草的 AI 识别，融合激光雷达与里程计完成语义 SLAM 建图，并负责全局任务调度与本地 AI 辅助分析报告；STM32 通过串口数据网桥接收指令，精确控制底盘、云台、靶向喷药和土壤采样；PC 端 Web 地面站则提供实时视频监控、地图显示与农技助手服务。

作业过程中，系统通过摄像头识别甘蔗、杂草等目标，结合激光雷达点云与里程计估计目标位置，将车辆轨迹、雷达点云、作物点、杂草点和土壤采样点叠加，构建出可查看、可追溯、可扩展的数字化语义地图，形成从现场巡检到数据资产的闭环。针对杂草，提出中心目标筛选、雷达侧向点匹配与已喷点记录组合策略，精准判断杂草相对车体的方位，触发对应方向机械臂实施靶向喷药，有效避免同一区域重复喷施。系统最终实现了实时视频、激光建图、语义标绘、全自动巡检及定向喷药精准除草、土壤数据归档及原始报告与专家报告生成等功能，为甘蔗地数字化管护提供了一种全自动、低成本、可落地的嵌入式智能装备方案。

第一部分 作品概述

1.1 功能与特性

本作品面向甘蔗地数字化管护场景，重点实现“看得见、标得准、管得住、能执行”的一体化作业流程。系统通过摄像头识别甘蔗与杂草，通过激光雷达和底盘里程计建立实时二维地图，将机器人运动轨迹、雷达点云、甘蔗点位、杂草点位和土壤采样结果叠加发送到 Web 云端中，形成数字化甘蔗地语义地图。用户可以远程查看视频，观察车辆在田间的运行轨迹和目标分布情况，便于后期数字化管理和复盘。

在执行层，系统支持全自动控制、自动化巡检、喷药总开关、左右视角切换、土壤检测、地图清空，任务录像数字化归档和特殊情况手动控制。杂草识别后，系统依据雷达侧向点位判断目标方位，驱动机械臂执行定向喷药，并通过已喷点记录避免重复喷施。PC 端本地服务器负责接收视频、报告和专家分析结果，使系统具备“边缘计算 + 本地服务器”的端边协同能力。

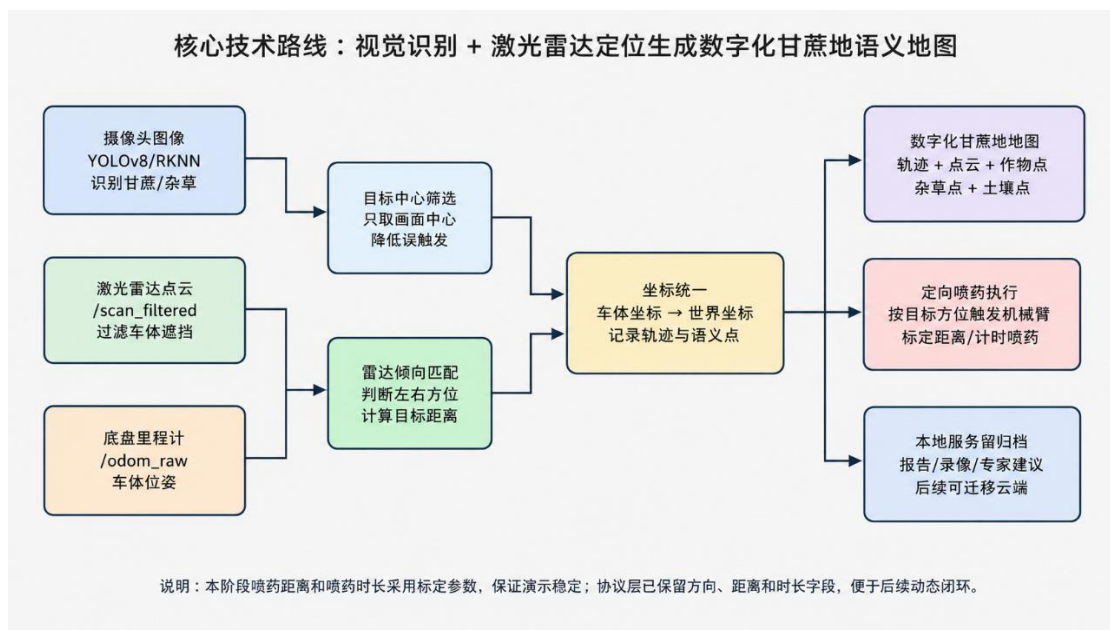


图 1-1 视觉识别与激光雷达融合生成数字化甘蔗地语义地图

1.2 应用领域

本作品主要应用于南方丘陵地区甘蔗田间管护场景，尤其适合小块分散、垄沟明显、人工巡检成本高、杂草分布不均的蔗田。系统能够将巡检过程转化为带空间位置的数字化数据，用于记录甘蔗长势、杂草分布、土壤采样点和作业轨迹，辅助农户判断哪些区域杂草压力较大、哪些位置需要复查或补喷。

除甘蔗地外，该平台也可迁移至玉米、烟草、蔬菜苗期等行作物场景。对于教学和科研，该系统可作为嵌入式边缘 AI、ROS2 建图、视觉-雷达融合、数据网桥、机器人状态机和农业数字化管理的综合实验平台。



图 1-2 蔗田人工巡检环境示意图



图 1-3 蔗田杂草两次大规模爆发情况图

1.3 主要技术特点

(1) 全自动巡航检测：系统可依据预设行长、行距、总行数和转向方向执行巡检，结合 /odom_raw 位移判断行程与调头节点，巡检过程中联动视觉识别、雷达建图和喷药状态机，实现无人干预下的连续检测与作业。

(2) 视觉-激光雷达语义地图：将 YOLOv8 识别结果与雷达/里程计坐标关联，生成作物点、杂草点、车辆轨迹、雷达点云和土壤采样点的统一地图，实现从单帧识别到空间化地块数据的转化。

(3) 侧向点位定向喷药和土壤检测：利用雷达侧向点判断杂草目标相对车体方位，配合机械臂完成左右方向定向喷药；同时通过土壤检测模块采集温湿度及氮、磷、钾等数据，为作业报告和农情评估提供依据。

(4) **数字化任务资产自动归档**: 任务结束后自动保存原始视频、AI 标注视频、语义地图、土壤数据、原始报告和本地 Qwen 专家报告, 便于赛后展示、农户复盘、历史追溯和后续云端迁移。

1.4 主要性能指标

表 1-1 主要性能指标

指标类别	当前实现/程序参数	说明
边缘视觉推理	RKNN YOLOv8 摄像头输入 640×480	甘蔗、杂草和病害类识别
雷达建图链路	/scan → filter.py → /scan_filtered → Cartographer	滤除车体遮挡区域后参与建图
数字地图图层	车辆轨迹、点云、 语义目标点、土壤点	甘蔗地数字化管理与复盘
轨迹记录周期	任务运行且运动时 约 0.2 s 记录一次	可视化行驶轨迹
雷达显示范围	盲区 0.15 m, 远界可设, 上传点云默认 4 m 内	Web 端可配置显示参数
喷药控制接口	方向左右, 时长 1-5 s,	当前距离采用 25 cm 标定值
巡检状态机	默认行长 6 m、行距 0.70 m、 总行数 4	Web 端可修改参数
本地资产同步	报告、原始视频、AI 标注视频 TCP 同步至 PC	PC 端作为本地服务器归档

1.5 主要创新点

(1) **全自动巡航检测**: 系统支持按行长、行距、总行数和转向方向自动巡检, 可在甘蔗地中完成连续巡航、行尾调头和巡检恢复, 减少人工遥控依赖。

(2) **视觉-激光雷达语义地图**: 将 YOLOv8 识别结果与激光雷达、里程计数据融合, 生成包含轨迹、点云、作物点、杂草点和土壤点的数字化语义地图。

(3) **侧向定向喷药和土壤检测**: 系统根据视觉目标和雷达侧向点判断杂草方位, 驱动机械臂定向喷药, 同时采集土壤温度、水分和氮磷钾数据, 实现除草与农情检测结合。

(4) **自动化数据归档分析**: 任务结束后自动保存原始视频、AI 标注视频、巡检数据、土壤数据和作业报告, 并生成专家分析结果, 便于后续复盘和数字化管理。

1.6 设计流程

设计流程遵循“需求痛点分析—硬件平台搭建—ROS2 建图链路—视觉模型部署—视觉雷达坐标关联—定向喷药状态机—PC 地面站与数据归档—现场联调优化”的路线，优先解决底盘、雷达、摄像头和串口通信的可用性，再逐步实现语义地图、喷药控制和本地服务器归档，最终形成可演示、可扩展的甘蔗地数字化作业系统。

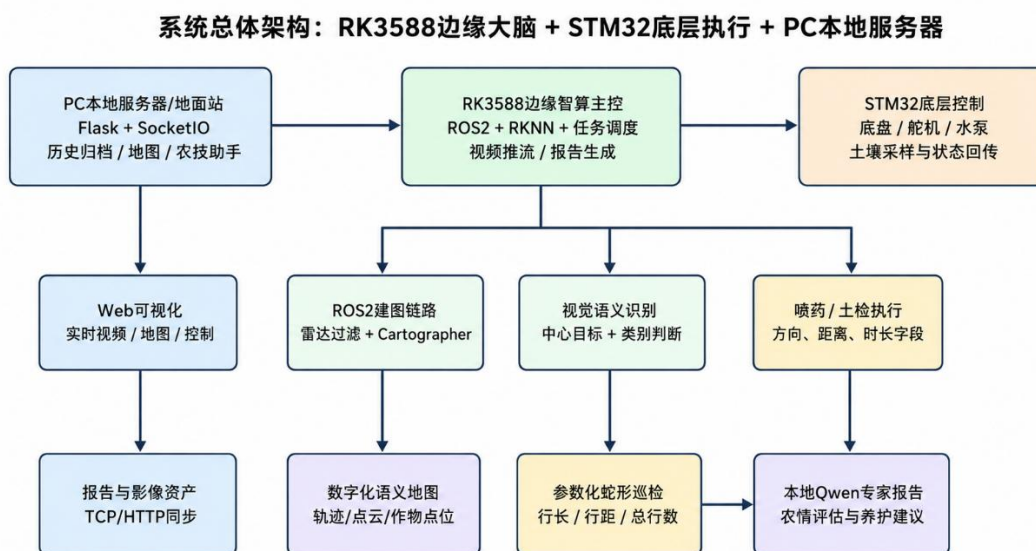


图 1-3 系统总体架构与模块关系

第二部分 系统组成及功能说明

2.1 整体介绍

系统整体围绕“感知—决策—执行—控制—归档分析”五大核心模块构建：

（1）多源环境感知模块通过激光雷达、摄像头和 IMU 获取田间环境、目标识别和车辆姿态信息；

（2）边缘智算决策模块以 RK3588 为核心，完成 AI 推理、SLAM 建图、语义标绘和任务调度；

（3）底层运动执行模块以 STM32 和底盘机构为基础，负责车辆运动、定点作业、喷药执行和土壤检测；

(4) 云端协同控制模块通过 PC 网页端实现任务下发、状态监控、人工接管和可视化管理；

(5) 数据归档分析模块则对原始视频、AI 标注视频、地图轨迹、检测数据和作业报告进行自动保存与分析，最终形成集自动巡检、智能识别、精准执行和数字化管理于一体的甘蔗地智慧农业作业系统。

系统五大核心模块关系框图

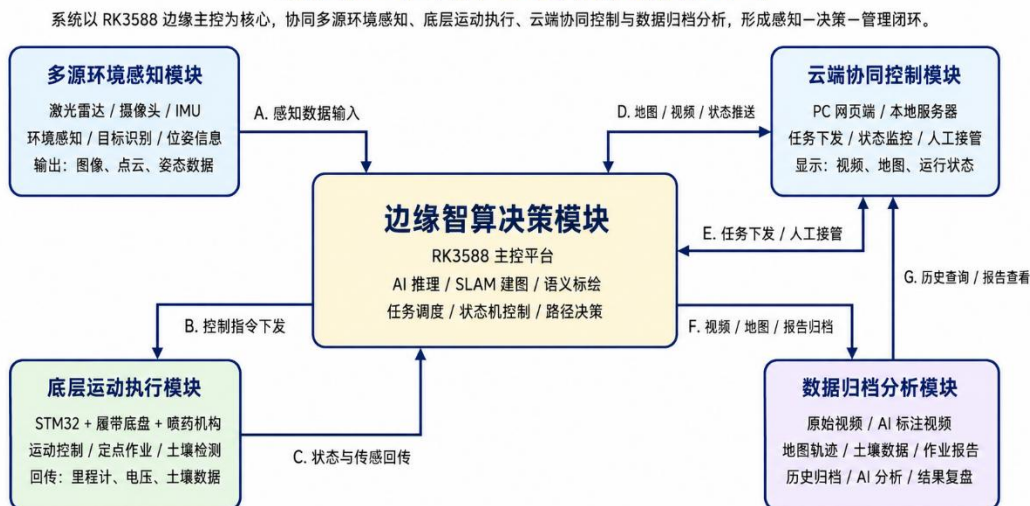


图 2-1 系统五大模块整体关系图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件系统以 RK3588 边缘计算板作为上层核心，配合 STM32 控制板实现底层实时执行。传感器包括摄像头、RPLidar 激光雷达、底盘里程计、电池电压采样和土壤温湿度/氮磷钾模块。执行机构包括履带底盘、云台、喷药机械臂、继电器水泵和土壤采样机构。

系统供电采用电池统一供电，经降压模块分别为主控、传感器、电机和舵机供电。RK3588 与 STM32 之间通过串口通信（dma），STM32 上传 20 字节定长数据帧，RK3588 下发底盘、视角、土检和喷药控制指令。

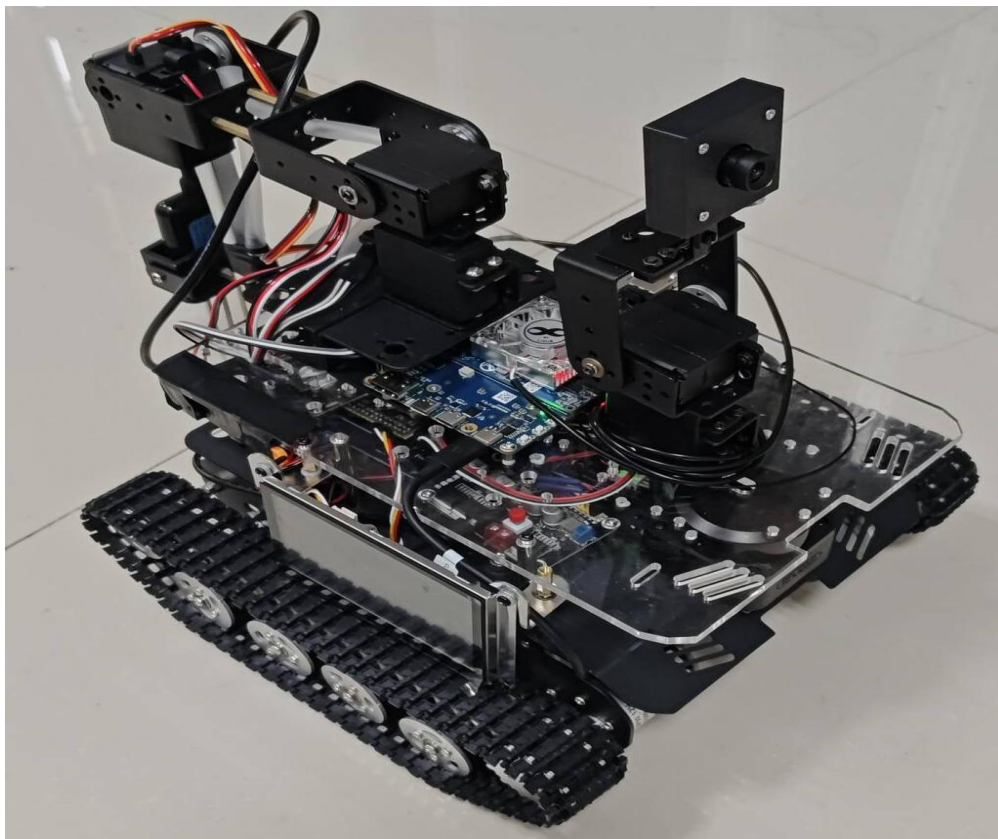


图 2-2 作业机器人整体实物

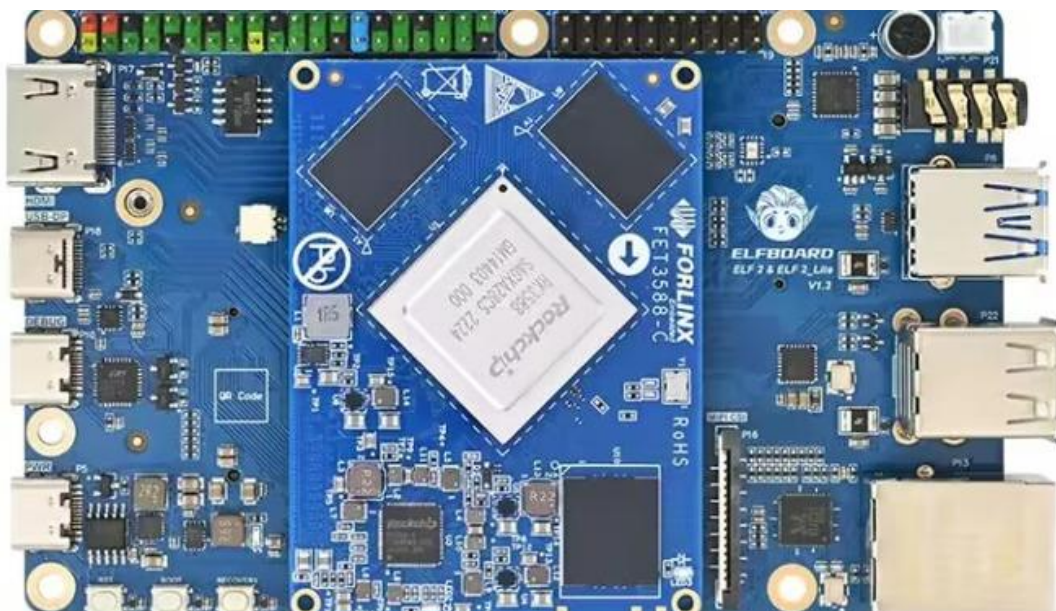


图 2-3 作业机器人整体实物

2.2.2 机械设计介绍：

机械结构以履带底盘为基础，能够适应泥土、垄沟和轻度起伏地面。激光雷达安装于车体中部，便于获得周向环境信息。软件中专门设计了雷达过滤节点，

将无效遮挡区域置为无穷大，避免其影响建图和前端显示。

喷药机械臂采用多舵机结构，用于完成喷头方位调整和定向喷施，喷头工作距离采用标定常量，喷药时长采用程序可控的计时方案，将雷达估算距离映射到舵机角度和药量时长，即可形成距离闭环喷药。土壤采样机构用于左右侧定点检测，将数据纳入任务报告。

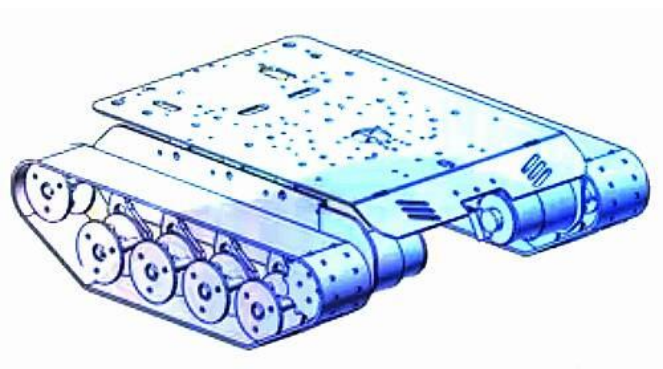


图 2-4 履带式行走底盘示意图

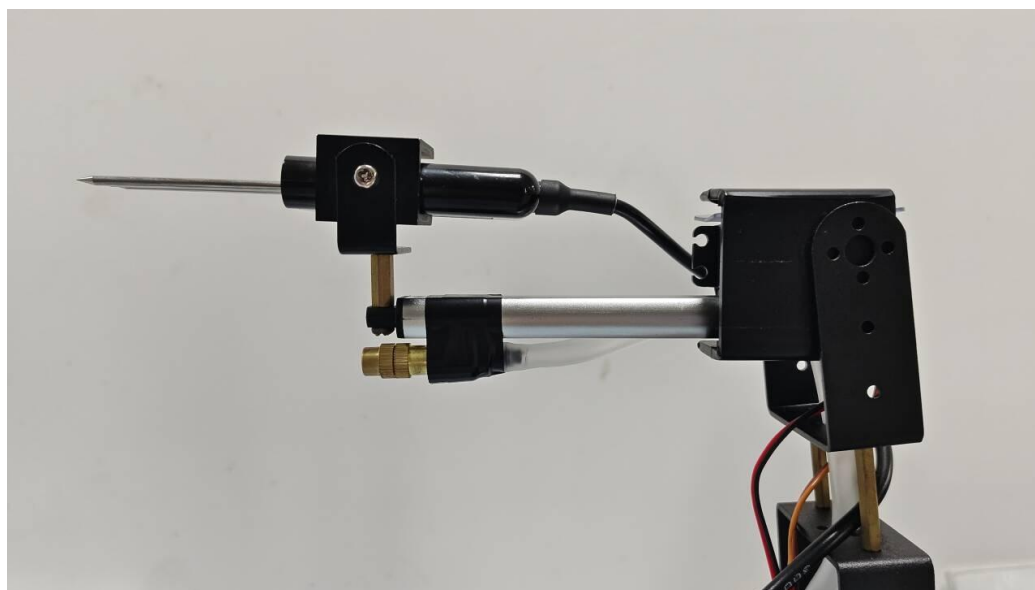


图 2-5 机械臂与定向喷药执行机构

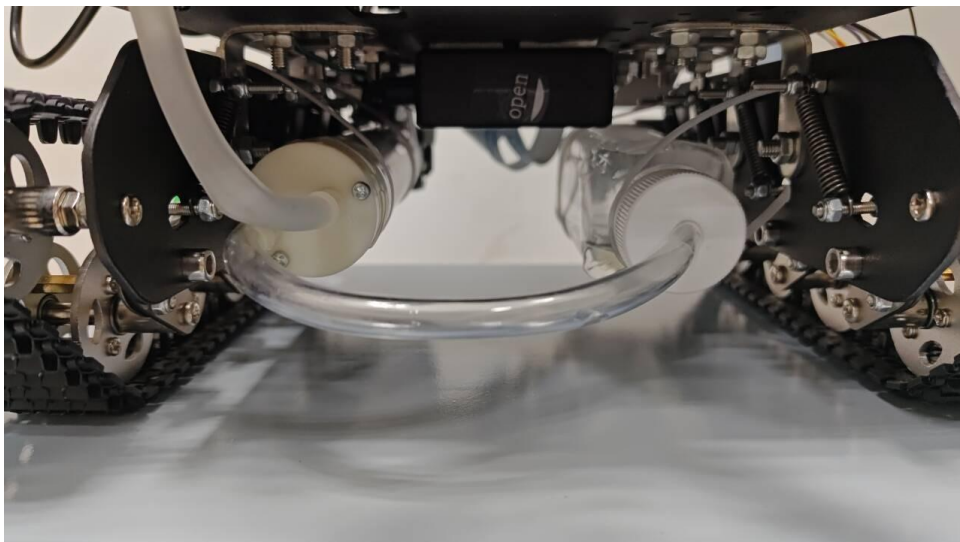


图 2-6 土壤采样与检测机构

2.2.3 电路各模块介绍：

电路系统包括主控供电、STM32 控制板、电机驱动、舵机接口、继电器水泵接口、传感器接口和串口通信接口。底层控制板完成电机、舵机和继电器的实时驱动，传感器数据通过统一串口链路上传至 ROS2 数据网桥。

为减少接口占用，系统在同一串口链路中复用两类数据帧：AA55 帧用于速度、角速度、航向角和电池电压；BB2A 帧用于土壤温度、水分、氮、磷、钾数据。ROS2 的 robot_node 采用滑动窗口查找帧头并完成分流，分别发布 /odom_raw、/battery_voltage 和 /stm_data 话题。

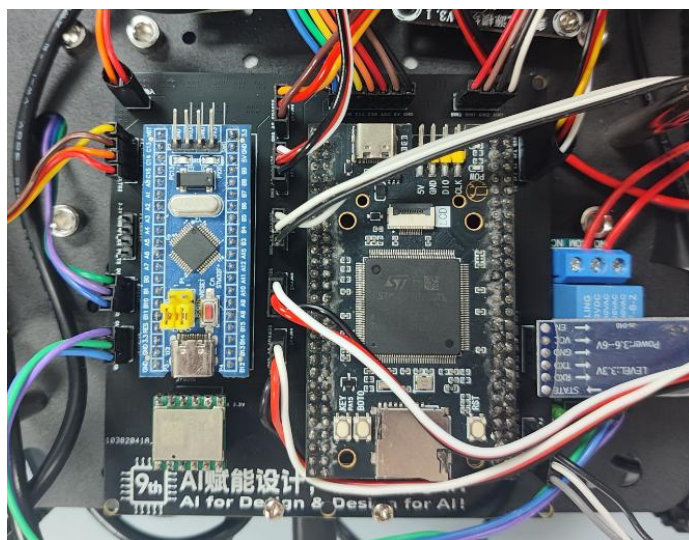


图 2-7 控制板与接口实物



图 2-8 串口数据帧分流设计

2.3 软件系统介绍

2.3.1 软件整体介绍（含 PC 端或云端，结合关键图片）；

软件系统由 RK3588 端、ROS2 节点层、自动巡检状态机、PC 服务器端和 Web 前端组成。RK3588 主控程序负责视频采集、RKNN 推理、Socket 视频遥测推送、串口指令下发、任务录像、语义标记和报告生成；ROS2 层负责雷达驱动、雷达过滤、底盘数据网桥和 Cartographer 建图；PC 端 server 作为 Flask + SocketIO 本地服务器，负责自动发现 RK3588、接收实时流、下发控制命令、接收多媒体资产和提供历史文件浏览。

Web 前端包含“全景控制大盘”和“智算视觉建图控制中心”两个页面。前者侧重历史资产、土壤曲线和农技助手；后者侧重实时视频、地图、手动控制、喷药开关、外设动作和巡航参数配置。

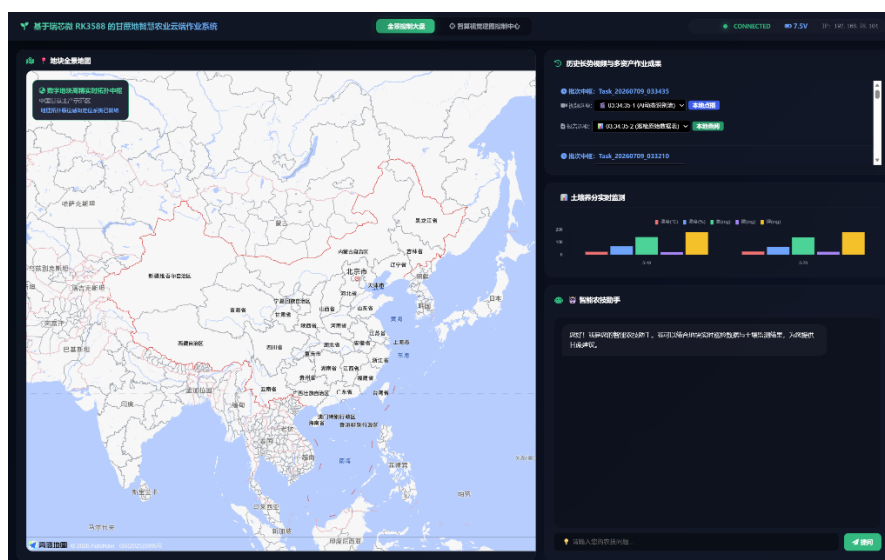


图 2-9 PC 数字化归档界面

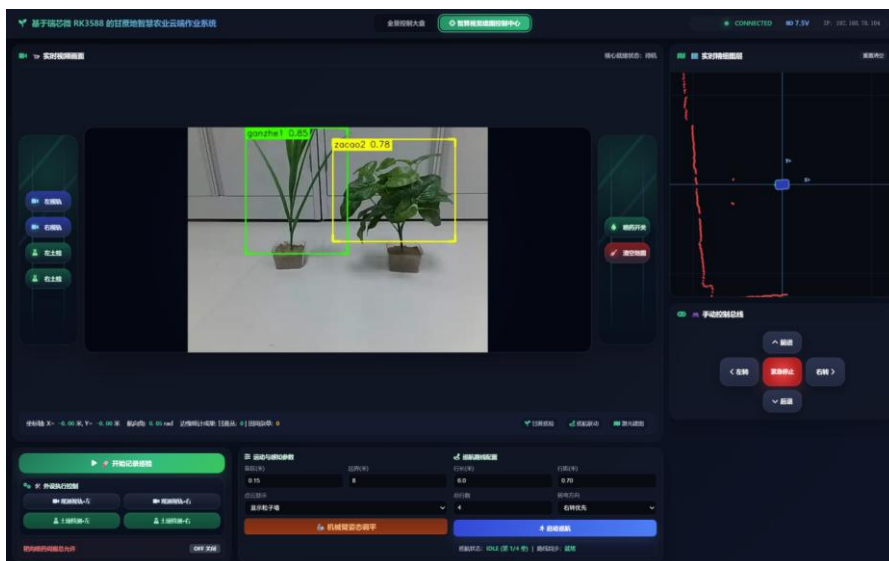


图 2-10 PC 本地服务器 Web 管理界面

2.3.2 软件各模块介绍：

(1) ROS2 建图与雷达过滤模块。

RPLidar 原始 `/scan` 数据进入 `filter.py`，程序将车体遮挡区域的点云置为无穷大后发布 `/scan_filtered`。Cartographer 使用过滤后的点云和 `/odom_raw` 进行二维建图与位姿估计，RK3588 主控订阅相关数据并推送到 Web 端。当前系统主要采用 Cartographer 与底盘里程计链路，保证稳定运行的情况下满足巡航任务。

ROS2建图与底层数据网桥流程

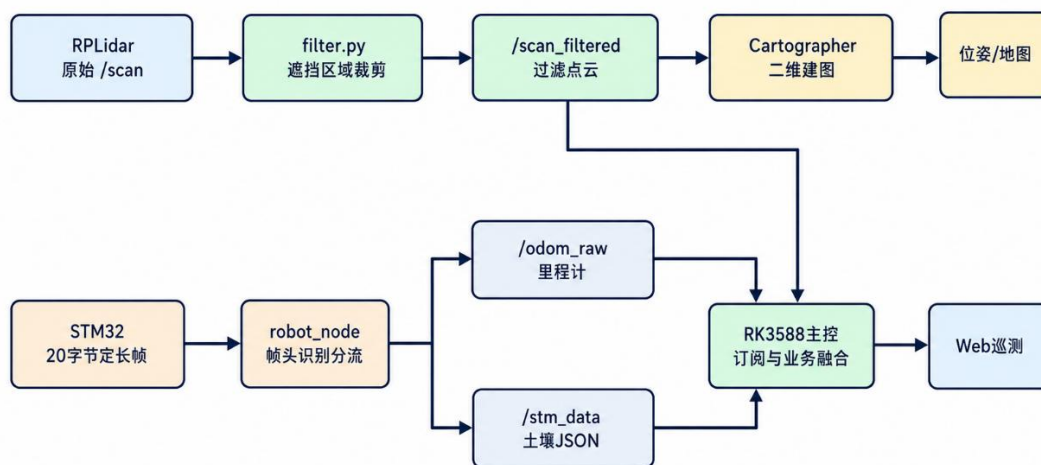


图 2-11 ROS2 建图与数据网桥流程

(2) 视觉-雷达语义地图模块。

YOLOv8 模型在 RK3588 上通过 RKNN 推理，主控程序优先选择画面中心目标，以减少边缘误识别和多目标干扰。系统根据当前车辆位姿，将雷达侧向匹配点转换到世界坐标中，形成甘蔗、杂草和土壤采样点等语义标记。每个标记包含类别、时间、坐标和相对车体方位，实现可追溯的数字化地块管理。

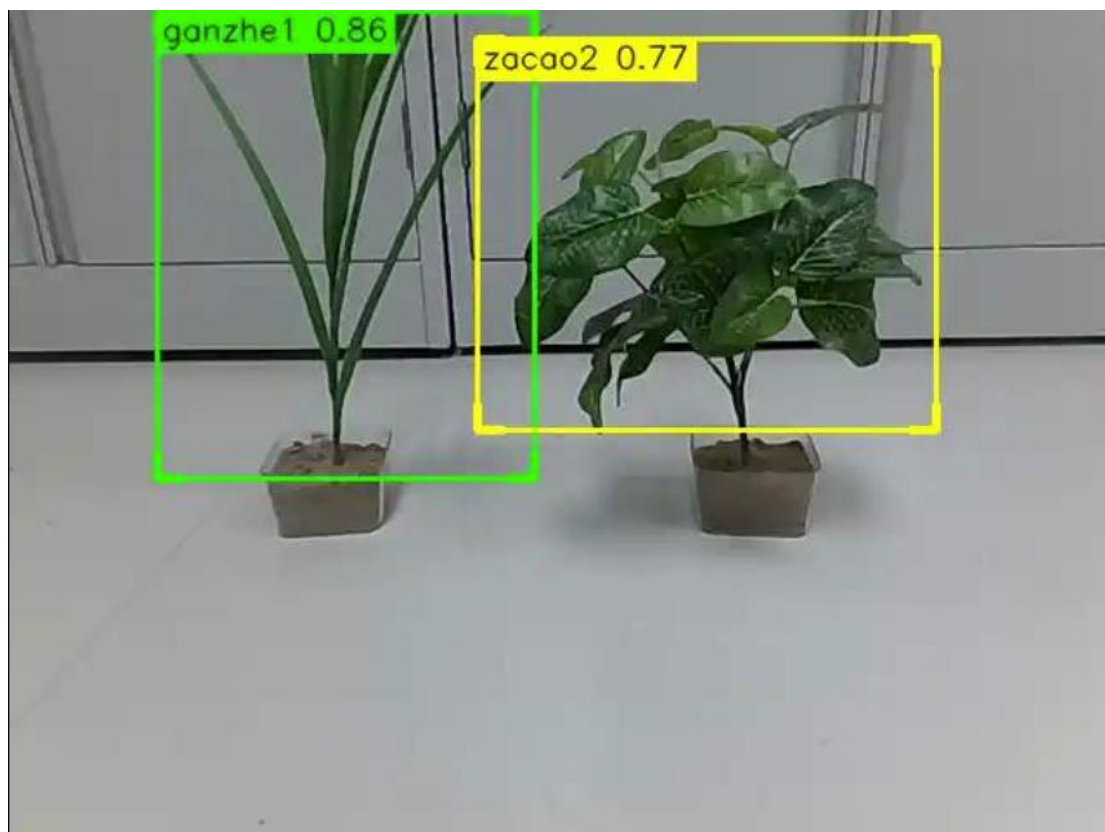


图 2-12 视觉检测与语义目标标绘效果

(3) 定向喷药控制模块。

系统检测到中心区域杂草后，先判断喷药总开关是否允许，再结合雷达侧向点确定目标方位。触发后车辆进入喷药阻塞状态，主控下发停车命令和喷药命令，并将目标点加入已喷队列，避免同一区域重复喷施。

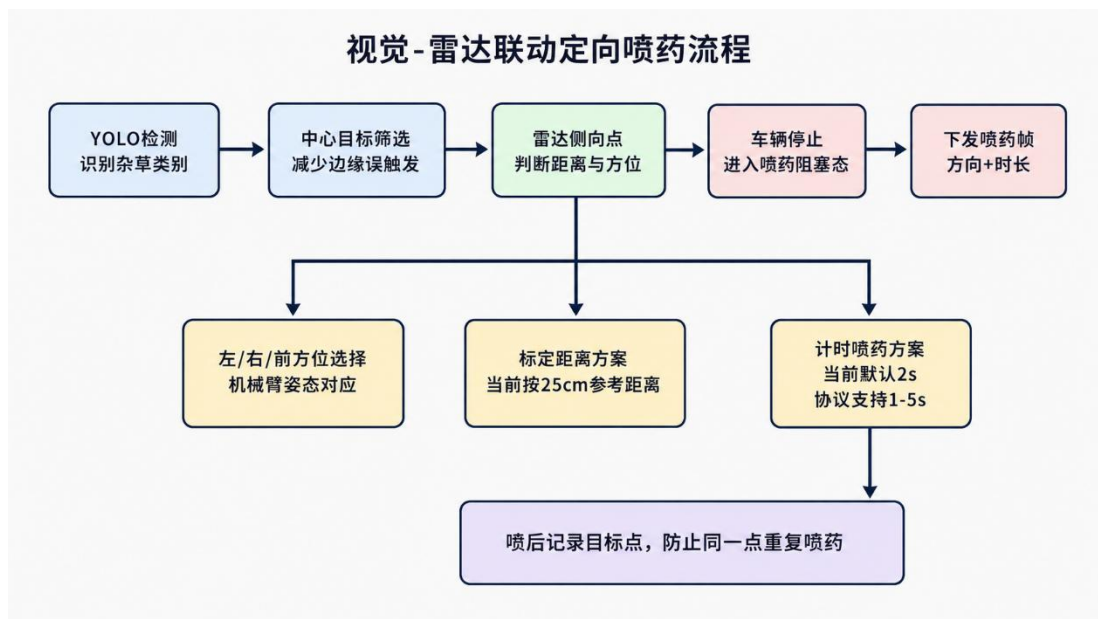


图 2-13 视觉-雷达联动定向喷药流程

(4) 全自动巡检模块。

aotu.py 作为独立状态机，与主推理线程解耦运行。用户可在 Web 端设置行长、行距、总行数和转向方向，状态机根据 /odom_raw 位移判断当前甘蔗地是否走完，到垄尾后执行调头动作。若巡检过程中触发喷药，状态机暂停底盘控制，待喷药完成后恢复巡检，从而避免运动和喷药控制冲突。

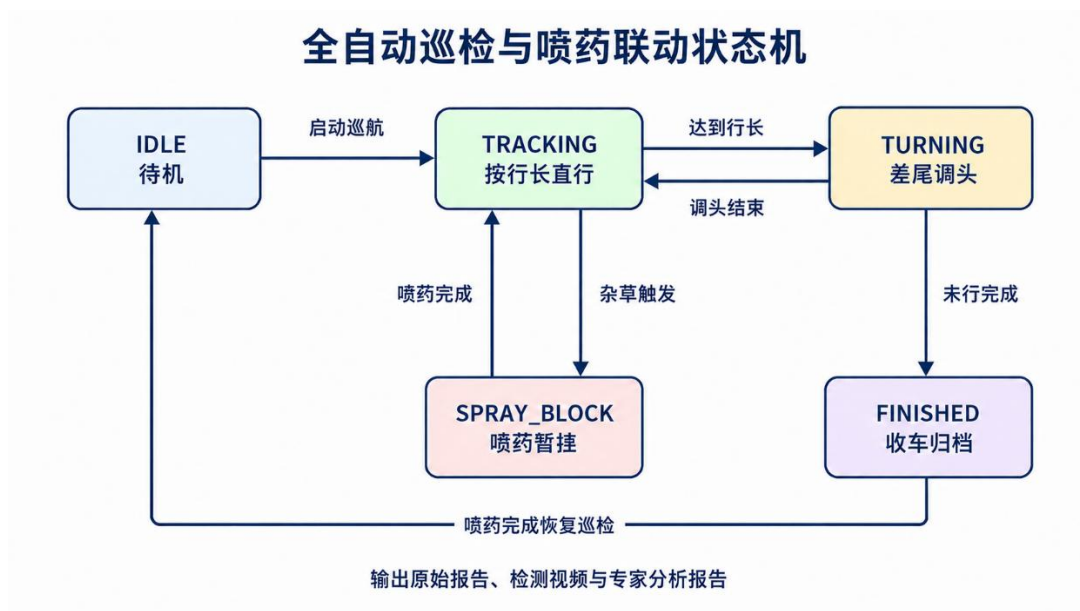


图 2-14 全自动巡检与喷药联动状态机

(5) 本地服务器与报告模块。

PC 端作为本地服务器接收实时视频和遥测数据，并通过 HTTP/TCP 接口

接收任务报告、原始视频和 AI 标注视频。任务结束后，RK3588 先生成原始报告，再启动本地 Qwen 模型脚本读取原始数据，输出面向农户的专家分析报告。该设计使系统在无公网条件下也能完成数据闭环，保留后续迁移至云服务器的能力。

（6）RK3588 模块

● RK3588 边缘智算主控模块：

该模块是 RK3588 端的总控程序，负责摄像头采集、RKNN 模型推理、ROS2 雷达/里程计/土壤数据订阅、Web 指令接收、串口下发 STM32 控制命令、语义地图标绘、定向喷药触发、视频遥测推送和任务报告生成，是整个系统的边缘智算调度中心。

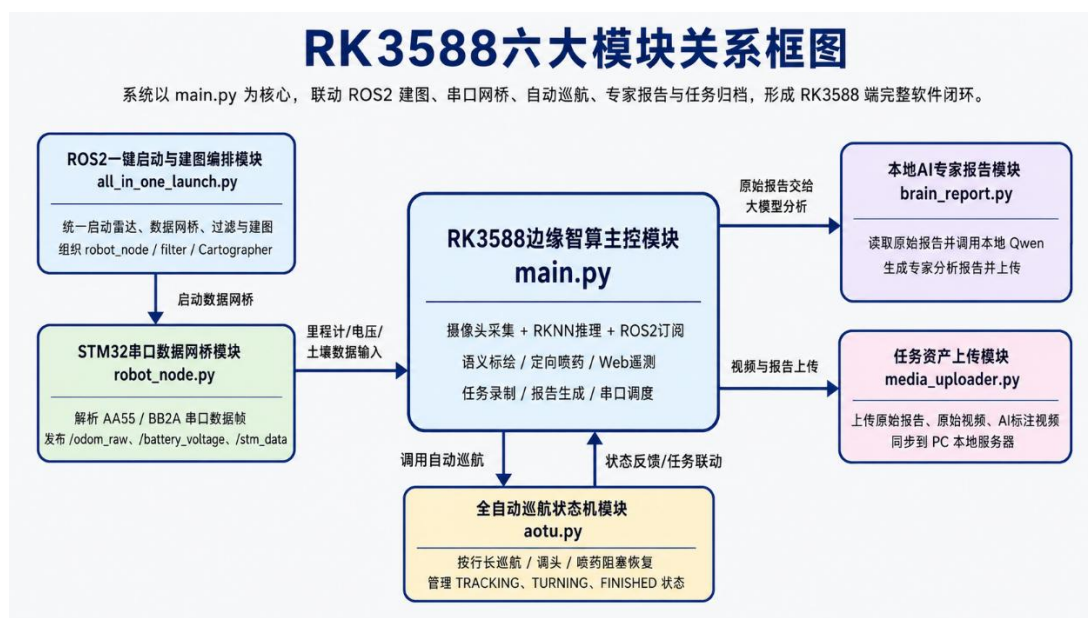


图 2-15 RK3588 模块关系图



```
# RK3588 边缘智算主控核心：摄像头推理 + ROS数据融合 + 语义标绘 + 喷药控制
# 1. 初始化 RKNN 推理线程池
pool = rknnPoolExecutor(
    rknnModel="./rknnModel/best.rknn",
    TPES=3,
    func=myFunc
)
# 2. 读取摄像头图像
cap = cv2.VideoCapture("/dev/video21")
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
# 3. 主循环：图像推理并更新目标中心点
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        continue
    # 将当前帧送入 RKNN 模型
    pool.put(frame)
    result, flag = pool.get()
    if flag and isinstance(result, dict):
        frame_res = result["img"] # 带检测框图像
        detections = result.get("detections", []) # 检测目标列表
        # 只选取画面中心目标，降低边缘误触发
        cx, cls_id = master_service.select_center_target_from_detections(detections)
        master_service.latest_yolo_cx = cx
        master_service.latest_yolo_cls = cls_id
    # 打包视频帧 + 遥测数据，推送给 PC 网页端
    telemetry = master_service.get_current_telemetry_snapshot()
    master_service.compound_asset_queue.put_nowait((frame_res, telemetry))
    # 视觉-雷达侧向匹配与定向喷药核心
    cx, cls_id = self.latest_yolo_cx, self.latest_yolo_cls
    # 只处理画面中心区域目标
    if 200 <= cx <= 440 and cls_id != -1:
        label_name = self.CLASSES_NAMES[cls_id]
        is_weed = "杂草" in label_name
        for i, distance in enumerate(self.real_lidar_data):
            if self.lidar_min_dist < distance < 0.80:
                angle = self.angle_min + i * self.angle_increment
                # 雷达局部坐标
                lx = distance * math.cos(angle)
                ly = -distance * math.sin(angle)
                # 侧向点筛选：判断目标在车身左侧或右侧
                if abs(lx) < 0.35 and abs(lx) / abs(ly) < 0.26:
                    wx = self.car_world_x + lx * math.cos(self.car_yaw) - ly * math.sin(self.car_yaw)
                    wy = self.car_world_y + lx * math.sin(self.car_yaw) + ly * math.cos(self.car_yaw)
                    # 避免重复喷药
                    if is_weed and not self.has_sprayed_weed_near(wx, wy):
                        self.safe_serial_send(self.CMD_STOP) # 先停车
                        direction = 2 if ly > 0 else 1 # 右侧或左侧喷药
                        self.send_spray_command(direction, 2) # 喷药 2 秒
                        self.mark_weed_sprayed(wx, wy)
                    # 写入语义地图点
                    self.semantic_markers.append([wx, wy, cls_id, label_name])
                    break
```

● STM32 数据网桥模块：

该模块是 RK3588 与 STM32 之间的 ROS2 数据桥。它通过串口读取 STM32 上传的 20 字节定长帧，并按帧头分流：AA55 帧解析为里程计、电压并发布 /odom_raw、/battery_voltage；BB2A 帧解析为土壤温度、水分、氮磷钾并发布 /stm_data，为建图、巡检和报告归档提供底层数据。

```
# STM32 数据网桥核心：AA55 里程计帧 + BB2A 土壤帧分流
class RobotNode(Node):
    def __init__(self):
        super().__init__('robot_node')
        # 发布 ROS2 话题
        self.odom_pub = self.create_publisher(Odometry, 'odom_raw', 10)
        self.vcc_pub = self.create_publisher(Float32, 'battery_voltage', 10)
        self.stm_data_pub = self.create_publisher(String, 'stm_data', 10)
        # 打开 STM32 串口
        self.ser = serial.Serial('/dev/ttyS9', 115200, timeout=0)
        self.packet_size = 20
        self.buffer = bytearray()
        self.robot_x = 0.0
        self.robot_y = 0.0
        self.last_time = None
        # 100Hz 读取串口
        self.timer = self.create_timer(0.01, self.timer_callback)

    def timer_callback(self):
        # 读取串口缓存
        self.buffer.extend(self.ser.read(self.ser.in_waiting))
        while len(self.buffer) >= self.packet_size:
            packet = self.buffer[:20]
            # BB2A: 土壤数据帧
            if packet[0] == 0xBB and packet[1] == 0x2A and packet[16] == 0x01:
                temp, moisture, n, p, k = struct.unpack('<ffHHH', packet[2:16])
                msg = String()
                msg.data = json.dumps({
                    "data_type": "soil_fertility",
                    "temp": round(temp, 2),
                    "moisture": round(moisture, 2),
                    "n": n,
                    "p": p,
                    "k": k
                })
                self.stm_data_pub.publish(msg)
                del self.buffer[:20]
                continue
            # AA55: 里程计与电压帧
            if packet[0] == 0xAA and packet[1] == 0x55:
                _, linear_vel, angular_vel, yaw_deg, vcc, crc, tail = struct.unpack('<HffffBB', packet)
                # 发布电池电压
                vcc_msg = Float32()
                vcc_msg.data = float(vcc)
                self.vcc_pub.publish(vcc_msg)
                # 速度积分得到 odom 坐标
                now = self.get_clock().now()
                yaw = math.radians(yaw_deg)
                odom = Odometry()
                odom.header.stamp = now.to_msg()
                odom.header.frame_id = 'odom'
                odom.child_frame_id = 'base_link'
                odom.twist.twist.linear.x = linear_vel
                odom.twist.twist.angular.z = angular_vel
                self.odom_pub.publish(odom)
                del self.buffer[:20]
            else:
                # 坏字节丢弃，继续滑动查找帧头
                del self.buffer[0]
```




- 全自动巡航状态机模块：

该模块负责甘蔗地自动巡航。用户可设置行长、行距、总行数和转向方向，状态机按 TRACKING → TURNING → FINISHED 的流程执行巡航、调头和收车；当主程序处于喷药阻塞状态时，巡航状态机会主动让出底盘控制权，待喷药完成后继续运行。

```
# 全自动巡航状态机核心
class TxtRouteExecutor:
    def __init__(self, master_robot, default_route_path):
        self.master = master_robot
        # 巡航参数，可由 Web 端修改
        self.row_length = 6.0
        self.row_spacing = 0.70
        self.total_rows = 4
        self.turn_dir = 1
        # 状态机状态
        self.current_row_index = 0
        self.sub_state = "IDLE"
        self._is_running = False
    def start(self, row_length, row_spacing, total_rows, turn_dir):
        # 启动巡航任务
        self.row_length = row_length
        self.row_spacing = row_spacing
        self.total_rows = total_rows
        self.turn_dir = turn_dir
        self.current_row_index = 0
        self.sub_state = "TRACKING"
        self._is_running = True
        # 联动主程序开启录像、轨迹和报告记录
        self.master.start_task()
    def _main_route_loop(self):
        start_x, start_y = 0.0, 0.0
        is_init_step = True
        while True:
            if not self._is_running:
                time.sleep(0.1)
                continue
            # 喷药时暂停巡航，避免抢占底盘
            if self.master.is_spraying_blocking:
                time.sleep(0.05)
                continue
            x = self.master.car_world_x
            y = self.master.car_world_y
            if self.sub_state == "TRACKING":
                if is_init_step:
                    start_x, start_y = x, y
                    is_init_step = False
                # 根据里程计判断当前垄是否走完
                distance = math.hypot(x - start_x, y - start_y)
                if distance < self.row_length:
                    self.master.safe_serial_send(bytes.fromhex("AA 02 01"))
                else:
                    if self.current_row_index + 1 < self.total_rows:
                        self.current_row_index += 1
                        self.sub_state = "TURNING"
                        is_init_step = True
                    else:
                        self.sub_state = "FINISHED"
            elif self.sub_state == "TURNING":
                # 根据转向方向执行车尾调头
                turn_cmd = "AA 08 01" if self.turn_dir > 0 else "AA 09 01"
                self.master.safe_serial_send(bytes.fromhex(turn_cmd))
                time.sleep(2.5)
                self.sub_state = "TRACKING"
            elif self.sub_state == "FINISHED":
                # 全部巡检完成，停车并触发报告生成
                self.master.safe_serial_send(bytes.fromhex("AA 07 01"))
                self.master.stop_task()
                self._is_running = False
                self.sub_state = "IDLE"
            time.sleep(0.05)
```

● ROS2 一键建图启动模块：

该模块是 ROS2 建图链路的一键启动脚本，负责统一启动串口权限配置、RPLidar 雷达驱动、STM32 数据网桥节点、雷达过滤脚本、静态 TF 变换和 Cartographer 建图节点，并将 Cartographer 的输入重映射到 /scan_filtered 和 /odom_raw，形成“雷达过滤 + 底盘里程计 + 二维建图”的完整链路。

```
# ROS2 一键建图启动核心：雷达 + 数据网桥 + 雷达过滤 + TF + Cartographer
def generate_launch_description():
    robot_driver_share_dir = get_package_share_directory('robot_driver')

    # 1. 串口权限配置
    set_permissions = ExecuteProcess(
        cmd=['sudo', 'chmod', '777', '/dev/ttyUSB0', '/dev/ttyUSB1', '/dev/ttyS9'],
        shell=True
    )

    # 2. 启动 RPLidar 雷达驱动
    rplidar_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('rplidar_ros'),
                'launch',
                'rplidar_c1_launch.py'
            )
        )
    )

    # 3. 启动 STM32 串口数据网桥
    robot_node = Node(
        package='robot_driver',
        executable='robot_node',
        name='robot_node',
        output='screen'
    )

    # 4. 启动雷达过滤脚本，输出 /scan_filtered
    filter_node = ExecuteProcess(
        cmd=['python3', os.path.expanduser('~/.ros2_ws/src/filter.py')],
        output='screen'
    )

    # 5. 发布 base_link 到 laser 的静态坐标变换
    static_tf_node = Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        arguments=['0', '0', '0.1', '0', '0', '0', 'base_link', 'laser']
    )

    # 6. 启动 Cartographer，使用过滤雷达和底盘里程计
    cartographer_node = Node(
        package='cartographer_ros',
        executable='cartographer_node',
        arguments=[
            '-configuration_directory', os.path.expanduser('~/.ros2_ws/src/robot_driver/launch'),
            '-configuration_basename', 'agri_slam.lua'
        ],
        remappings=[
            ('scan', '/scan_filtered'),
            ('odom', '/odom_raw')
        ]
    )

    return LaunchDescription([
        set_permissions,
        robot_node,
        rplidar_launch,
        filter_node,
        static_tf_node,
        cartographer_node
    ])

```

- 数字化任务资产上传模块：

该模块负责将一次任务结束后生成的原始报告、原始视频和 AI 标注视频上传到 PC 本地服务器。程序采用 TCP 方式先发送“文件名 + 文件大小”包头，再按 64KB 分块发送文件内容，实现 RK3588 到地面站的任务资产同步。

```
# 任务资产上传核心：报告 + 原始视频 + AI标注视频 TCP 传输

def blind_upload_file(file_path, pc_ip, port=9995):
    if not os.path.exists(file_path):
        return False

    file_name = os.path.basename(file_path)
    file_size = os.path.getsize(file_path)

    # 建立到 PC 端的 TCP 连接
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((pc_ip, port))

    # 先发送文件头：文件名和大小
    header = f"{file_name}:{file_size}\n".encode("utf-8")
    sock.sendall(header)

    # 再分块发送文件内容
    with open(file_path, "rb") as f:
        while True:
            chunk = f.read(65536)
            if not chunk:
                break
            sock.sendall(chunk)

    sock.close()
    return True

# 任务结束后上传三类核心资产
raw_video = os.path.join(save_dir, f"Video_Raw_{timestamp}.mp4")
box_video = os.path.join(save_dir, f"Video_Box_{timestamp}.mp4")

media_queue = [
    report_path, # 原始任务报告
    raw_video,   # 原始视频
    box_video    # AI 标注视频
]

for path in media_queue:
    blind_upload_file(path, pc_ip)
```

- 本地 AI 专家报告生成模块：

该模块负责在任务结束后读取 RK3588 生成的原始巡检报告，将土壤数据、目标识别结果和语义标绘信息组织成 Prompt，调用本地 Qwen2-VL/RKLLM 模型生成面向农户的专家分析报告，并将最终报告保存到任务目录后上传至 PC 端。

```
# 本地 Qwen 专家报告生成核心：读取原始报告 -> 调用本地大模型 -> 保存专家报告

# 1. 读取原始巡检报告
with open(raw_report_path, "r", encoding="utf-8") as f:
    raw_text = f.read()

# 2. 构造农业专家分析 Prompt
prompt = (
    "你是自动化农业机器人诊断专家。请根据以下巡检原始数据，"
    "生成一份面向农户的农业质检报告，内容包括：\n"
    "1. 甘蔗苗情与生长势头评估\n"
    "2. 杂草危害等级与空间分布分析\n"
    "3. 水肥追施和定向除草建议\n\n"
    "【巡检原始数据】:\n" + raw_text
)

# 3. 调用 RK3588 本地大模型
process = subprocess.Popen(
    [".//llm", "../Qwen2-VL-2B-Instruct.rkllm", "512", "2048"],
    cwd="/home/elf/Data/model/demo_Linux_aarch64/",
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True,
    encoding="utf-8"
)

stdout, stderr = process.communicate(input=prompt, timeout=45)

# 4. 清洗并保存专家报告
expert_report_path = os.path.join(save_dir, f"Expert_Report_{timestamp}.txt")

with open(expert_report_path, "w", encoding="utf-8") as f:
    f.write("Qwen 本地 AI 智库 - 农田巡检专家分析报告\n")
    f.write("=" * 50 + "\n")
    f.write(stdout.strip())

# 5. 上传到 PC 端报告接口
with open(expert_report_path, "rb") as f:
    requests.post(
        f"http://{pc_ip}:5000/api/upload-report",
        files={"file": f},
        timeout=15
    )
```


(7) STM32 底层执行与数据网桥模块。

STM32 底层程序主要负责履带底盘实时控制、编码器速度采集、JY901S 姿态读取、电池电压采样、喷药机械臂与土壤检测模块转发控制，以及与 RK3588 主控之间的串口通信。程序采用 FreeRTOS 进行任务划分，将电机闭环控制、后台状态维护和里程计数据上报分离，避免底盘控制、传感器采集和串口通信互相阻塞。

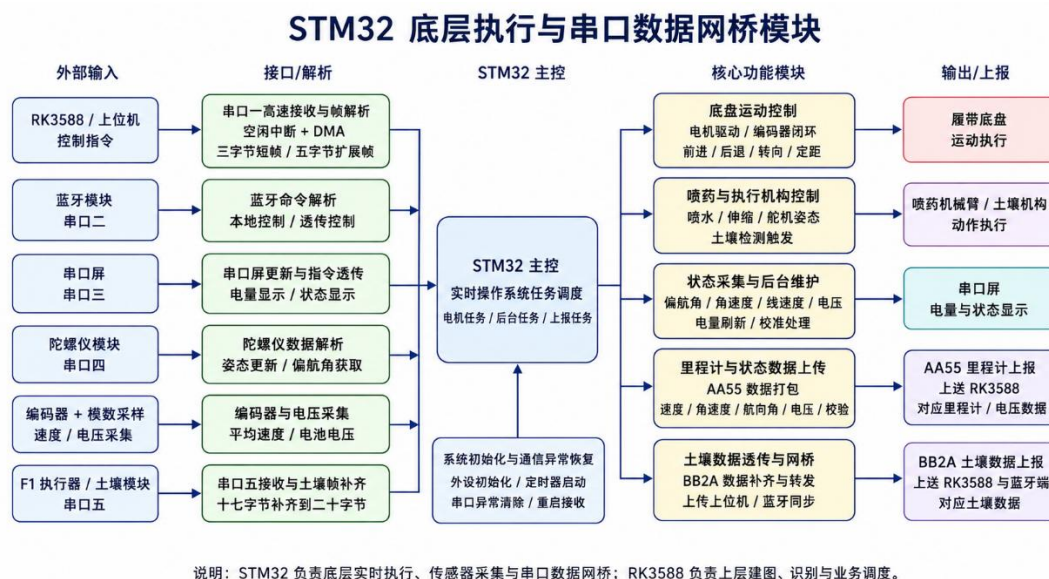


图 2-16 STM32 底层执行与串口数据网桥模块

● FreeRTOS 实时任务调度模块

该模块将底盘实时控制放在高优先级任务中周期执行，每 7 ms 调用一次 `Motor_Control_Task()`，保证电机 PWM 和编码器反馈能够稳定闭环。后台任务负责 IMU 处理、陀螺仪复位校准和电量刷新，降低对电机实时控制的影响。



```
/* 电机实时控制任务：高频闭环，保证底盘运动稳定 */
void MotorTask(void *argument)
{
    for(;;)
    {
        Motor_Control_Task(); // 读取编码器反馈并更新电机 PWM
        osDelay(7);           // 约 7ms 控制周期
    }
}

/* 后台维护任务：处理 IMU、电量显示和校准标志 */
void BackTask(void *argument)
{
    static uint16_t battery_timer = 0;

    for(;;)
    {
        IMU_Proc();           // 刷新陀螺仪数据

        if(res_flag)          // 收到陀螺仪复位/校准请求
        {
            IMU_Calibration_Feedback(0);
            osDelay(1000);
            JY901S_CaliYawZero();
            res_flag = 0;
        }

        if(++battery_timer >= 100) // 约 1s 更新一次电量
        {
            battery_timer = 0;
            float percent = (vcc - 7.1f) / (10.5f - 7.1f) * 100.0f;
            Screen_Update_Battery(percent);
        }

        osDelay(10);
    }
}
```

- 里程计、电压与姿态数据上传模块

该模块是 STM32 与 RK3588 建图链路的核心。STM32 将编码器平均速度、陀螺仪 Z 轴角速度、Yaw 航向角和电池电压打包成固定结构体，通过 USART1 周期上传给 RK3588。RK3588 端可进一步转换为 /odom_raw、/battery_voltage 等 ROS2 话题，为 Cartographer 建图、Web 状态显示和任务记录提供底盘基础数据，并在 DebugTask 中以 50 ms 周期连续上报。



```
#pragma pack(push, 1)
typedef struct {
    uint16_t header;           // 帧头: 0x55AA
    float    linear_velocity;  // 线速度, 单位 m/s
    float    angular_velocity; // 角速度, 单位 rad/s
    float    yaw;              // 航向角, 单位 °
    float    vcc;              // 电池电压
    uint8_t  crc;              // 累加和校验
    uint8_t  tail;             // 帧尾: 0x01
} RobotOdomPacket_t;
#pragma pack(pop)
// 里程计、电压、姿态数据 20Hz 上传
void DebugTask(void *argument)
{
    RobotOdomPacket_t tx_packet;
    tx_packet.header = 0x55AA;
    tx_packet.tail   = 0x01;
    for(;;)
    {
        IMU_Proc();
        // 采集电池电压
        adcx = Get_adc_Average(ADC_CHANNEL_2, 10);
        vcc = (float)adcx * (3.3f * 11.0f / 4096.0f);
        // 编码器速度 cm/s 转 m/s
        float raw_speed_m_s = Get_Average_Speed_CmS() / 100.0f;
        // 陀螺仪角速度 °/s 转 rad/s
        float raw_gyro_rad_s = fGyro[2] * 0.0174532925f;
        // 静止滤噪, 减少建图漂移
        if(fabsf(raw_speed_m_s) < 0.005f &&
            fabsf(raw_gyro_rad_s) < 0.002f)
        {
            tx_packet.linear_velocity = 0.0f;
            tx_packet.angular_velocity = 0.0f;
        }
        else
        {
            tx_packet.linear_velocity = raw_speed_m_s;
            tx_packet.angular_velocity = raw_gyro_rad_s;
        }
        tx_packet.yaw = get_yaw_with_zero();
        tx_packet.vcc = vcc;
        // 累加和校验
        uint8_t *ptr = (uint8_t *)&tx_packet;
        uint8_t checksum = 0;
        for(int i = 0; i < sizeof(RobotOdomPacket_t) - 2; i++)
        {
            checksum += ptr[i];
        }
        tx_packet.crc = checksum;
        HAL_UART_Transmit(&huart1,
            (uint8_t *)&tx_packet,
            sizeof(tx_packet),
            10);
        osDelay(50); // 20Hz 上传
    }
}
```

● RK3588 串口控制指令解析模块

该模块负责接收 RK3588 或上位机下发的控制命令，并转换为底盘、喷药、舵机、土壤检测等底层动作。短指令用于底盘运动，例如前进、后退、转向、停止和定距移动；扩展指令通过 UART5 透传给 F1/机械臂子控制器，实现喷水、伸缩、舵机姿态调整和土壤检测等操作。main.c 中 UART_Proc() 同时保留了底盘控制、本机状态查询和 F1 扩展控制三类指令。

```
// RK3588 下发的串口指令处理
void UART_Proc(uint8_t cmd)
{
    switch(cmd)
    {
        // ----- 底盘运动控制 -----
        case 0x02: // 前进
            Motor_SetSpeed(left_speed, -right_speed);
            break;
        case 0x03: // 后退
            Motor_SetSpeed(-left_speed, right_speed);
            break;
        case 0x04: // 左转 90°
            Turn_Left_90();
            break;
        case 0x05: // 右转 90°
            Turn_Right_90();
            break;
        case 0x06: // 原地 180° 调头
            Turn_180();
            break;
        case 0x07: // 停车
            Motor_SetSpeed(0, 0);
            break;
        case 0x10: // 前进 1m
            Motor_Move_Distance(left_speed, right_speed, 1000);
            break;
        case 0x11: // 前进 2m
            Motor_Move_Distance(left_speed, right_speed, 2000);
            break;
        // ----- 喷药与执行机构控制 -----
        case 0x21: // 喷水 1000ms
        case 0x22: // 喷水 2000ms
        case 0x23: // 喷水 3000ms
        case 0x24: // 伸缩动作
        case 0x25: // 舵机左转
        case 0x26: // 舵机回中/向后
        case 0x27: // 舵机右转
        case 0x2D: // 读取土壤传感器
        case 0x36: // 自动伸缩全流程
        case 0x37: // 左侧喷药全流程
        case 0x38: // 右侧喷药全流程
            HAL_UART_Transmit(&huart5, &cmd, 1, 50);
            break;
        default:
            UART1_Send_With_CRC("[Error] Unknown Cmd\r\n", 21);
            break;
    }
}
```

● USART1 高速 DMA 接收与帧解析模块

USART1 是 RK3588 与 STM32 之间的主要通信链路。程序使用 HAL_UARTEx_ReceiveToIdle_DMA() 进行空闲中断 DMA 接收，可以接收不定长数据，并在一帧数据发送完成后统一进入解析流程。解析逻辑以 0xAA 作为帧头，兼容两类指令：AA + cmd + 01 三字节本地控制帧，以及 AA + d1 + d2 + d3 + 01 五字节扩展帧。三字节帧由 STM32 本地执行，五字节帧直接转发给机械臂或喷药子控制器。

```
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    if(huart->Instance == USART1)
    {
        for(uint16_t i = 0; i < Size; i++)
        {
            if(uart1_rx_dma_buf[i] == 0xAA)
            {
                uart1_cnt = 0;
                memset(uart1_buf, 0, sizeof(uart1_buf));
                uint16_t j = i + 1;
                // 优先解析三字节短帧: AA cmd 01
                while(j < Size && uart1_cnt < 2)
                {
                    uart1_buf[uart1_cnt++] = uart1_rx_dma_buf[j++];
                }
                if(uart1_cnt == 2 && uart1_buf[1] == 0x01)
                {
                    UART_Proc(uart1_buf[0]);
                    i = j;
                    continue;
                }
                // 继续解析五字节扩展帧: AA d1 d2 d3 01
                while(j < Size && uart1_cnt < 4)
                {
                    uart1_buf[uart1_cnt++] = uart1_rx_dma_buf[j++];
                }
                if(uart1_cnt == 4 && uart1_buf[3] == 0x01)
                {
                    HAL_UART_Transmit(&huart5,
                                         &uart1_rx_dma_buf[i],
                                         5,
                                         50);
                    i = j;
                    continue;
                }
            }
        }
        // 重新开启 DMA 空闲接收
        memset(uart1_rx_dma_buf, 0, sizeof(uart1_rx_dma_buf));
        HAL_UARTEx_ReceiveToIdle_DMA(&huart1,
                                     uart1_rx_dma_buf,
                                     UART1_RX_BUF_SIZE);
    }
}
```


- 土壤检测与机械臂反馈数据透传模块

UART5 用于连接喷药机械臂、土壤检测或下级 F1 控制模块。当下级模块返回土壤肥力数据时，程序判断其是否为 BB 2A ... 01 格式的 17 字节土壤帧，并将其补齐为 20 字节定长帧后上传至 RK3588 和蓝牙端。这样上位机在接收时不需要处理变长数据，可以稳定区分里程计帧和土壤帧，便于 ROS2 数据网桥发布 /stm_data。

```
if(huart->Instance == UART5)
{
    HAL_UART_DMAStop(&huart5);

    // BB 2A 土壤肥力帧：17 字节补齐为 20 字节
    if(Size == 17 &&
        uart5_buf[0] == 0xBB &&
        uart5_buf[1] == 0x2A &&
        uart5_buf[16] == 0x01)
    {
        uint8_t aligned_buf[20] = {0};

        memcpy(aligned_buf, uart5_buf, 17);

        // 上传 RK3588，并同步给蓝牙调试端
        HAL_UART_Transmit(&huart1, aligned_buf, 20, 50);
        HAL_UART_Transmit(&huart2, aligned_buf, 20, 50);
    }
    else
    {
        // 非土壤帧按原样透传
        HAL_UART_Transmit(&huart1, uart5_buf, Size, 50);
        HAL_UART_Transmit(&huart2, uart5_buf, Size, 50);
    }

    memset(uart5_buf, 0, sizeof(uart5_buf));
    HAL_UARTEx_ReceiveToIdle_DMA(&huart5,
        uart5_buf,
        sizeof(uart5_buf));
}
```

- 系统初始化与通信异常恢复模块

系统启动后依次初始化 ADC、IMU、编码器、PWM、电机和各路 UART 接收，并开启 TIM8 定时器中断。USART1 采用 DMA 空闲接收，USART2、USART3、UART4 采用单字节中断接收，UART5 采用 DMA 空闲接收。为了避免串口噪声或大数据突发造成接收挂死，错误回调中会清除溢出和帧错误标志，并重新启动 USART1 DMA 接收。

```
// 外设与通信初始化
adc_Init();
JY901S_Init();
Encoder_Init();
PWM_Init();
Motor_SetSpeed(0, 0);

// 开启各路串口接收
HAL_UARTEx_ReceiveToIdle_DMA(&huart1,
                             uart1_rx_dma_buf,
                             UART1_RX_BUF_SIZE);

HAL_UART_Receive_IT(&huart2, &bluetooth_rx_byte, 1);
HAL_UART_Receive_IT(&huart3, &uart3_rx_data, 1);
HAL_UART_Receive_IT(&huart4, &uart4_imu_rx_data, 1);

HAL_UARTEx_ReceiveToIdle_DMA(&huart5,
                             uart5_buf,
                             sizeof(uart5_buf));

HAL_TIM_Base_Start_IT(&htim8);
```

(8) PC 端软件模块

- PC 前端 Web 地面站模块

PC 前端 Web 地面站主要负责把后端接收到的 RK3588 实时视频、车辆坐标、航向角、电池电压、巡航状态、语义标记、土壤数据和历史任务资产展示给用户，同时提供全自动巡检启动/停止、参数设置、地图清空、喷药开关和农技问答入口。前端本身不直接连接 RK3588，而是通过 SocketIO 接收后端推送的 video_frame 和 telemetry_update，并通过 HTTPAPI 向后端发送控制请求，后端再转发给 RK3588。



```
<!-- PC 前端 Web 地面站核心：接收视频、遥测数据，并下发控制指令 -->
<!-- 实时视频显示 -->
<img id="videoView" style="width:640px;height:480px;" />
<!-- 状态显示 -->
<div>
  坐标: <span id="posText">0, 0</span>
  航向角: <span id="yawText">0</span>
  电池: <span id="batteryText">0V</span>
  巡航状态: <span id="patrolText">待机</span>
</div>
<!-- 常用控制按钮 -->
<button onclick="sendCmd('FORWARD')">前进</button>
<button onclick="sendCmd('STOP')">停止</button>
<button onclick="sendCmd('CLEAR_MAP')">清空地图</button>
<button onclick="startPatrol()">启动全自动巡检</button>
<button onclick="stopPatrol()">停止巡检</button>
<script src="/socket.io/socket.io.js"></script>
<script>
// 建立与 Flask-SocketIO 后端的连接
const socket = io();
// 接收后端推送的视频帧
socket.on("video_frame", function(data) {
  document.getElementById("videoView").src =
    "data:image/jpeg;base64," + data.image;
});
// 接收后端推送的车辆遥测、地图和土壤数据
socket.on("telemetry_update", function(data) {
  document.getElementById("posText").innerText =
    data.car_world_x.toFixed(2) + ", " + data.car_world_y.toFixed(2);
  document.getElementById("yawText").innerText =
    data.car_yaw.toFixed(2);
  document.getElementById("batteryText").innerText =
    data.battery_voltage.toFixed(2) + "V";
  document.getElementById("patrolText").innerText =
    data.nav_running ? "巡检中" : "待机";
  // 这里可继续把 recorded_path、permanent_obstacles、
  // semantic_markers 绘制到地图画布中
});
// 向后端发送普通控制指令，由后端转发给 RK3588
function sendCmd(cmd) {
  fetch("/api/gcs/cmd/" + cmd, {
    method: "POST"
  });
}
// 启动全自动巡检
function startPatrol() {
  fetch("/api/gcs/start-patrol", {
    method: "POST"
  });
}
// 停止全自动巡检
function stopPatrol() {
  fetch("/api/gcs/stop-patrol", {
    method: "POST"
  });
}
// 设置巡检参数：行长、行距、总行数、转向方向
function setPatrolParams(rowLen, rowSpace, totalRows, turnDir) {
  fetch("/api/gcs/update-row-length", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({value: rowLen})
  });
  fetch("/api/gcs/update-row-spacing", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({value: rowSpace})
  });
  fetch("/api/gcs/update-total-rows", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({value: totalRows})
  });
  fetch("/api/gcs/update-turn-dir", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({value: turnDir})
  });
}
// 查询历史任务资产：原始视频、AI标注视频、报告等
function loadHistoryAssets() {
  fetch("/api/sugarcane/history-videos")
    .then(res => res.json())
    .then(data => {
      console.log("历史归档文件: ", data.videos);
    });
}
</script>
```

- PC 后端 Flask + SocketIO 服务模块

PC 后端是整个地面站的数据中转与资产管理中心，基于 Flask 提供 HTTP 接口，基于 SocketIO 向前端实时推送视频与遥测数据。它一方面通过 UDP 自动发现 RK3588 的 IP，通过 TCP 连接 RK3588 的实时流端口，接收“图像帧+JSON 遥测数据”的复合数据包，并将其解析为前端可显示的视频流、车辆坐标、轨迹、点云、语义目标、土壤数据和巡检状态；另一方面，它接收前端按钮或参数设置请求，再通过 TCP 指令端口转发给 RK3588。此外，后端还负责接收任务结束后上传的原始报告、原始视频、AI 标注视频和专家报告，并按任务时间戳自动归档到本地文件夹。



```
# PC 后端核心: Flask + SocketIO + RK3588实时流接收 + 指令转发 + 资产归档

# ===== 基础服务配置 =====
app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*", async_mode="eventlet")
RK_IP = "192.168.0.232" # RK3588 地址, 可由 UDP 自动发现更新
RK_PORT = 8888 # RK3588 视频 + 遥测复合流端口
RK_CMD_PORT = 8889 # RK3588 指令接收端口
ASSETS_RECV_PORT = 9995 # 任务资产接收端口
BASE_SAVE_PATH = r"C:\Users\26405\uploads"
os.makedirs(BASE_SAVE_PATH, exist_ok=True)
# 保存最新遥测数据, 供前端查询和农技助手使用
latest_data = {
    "car_world_x": 0.0,
    "car_world_y": 0.0,
    "car_yaw": 0.0,
    "recorded_path": [],
    "permanent_obstacles": [],
    "semantic_markers": [],
    "battery_voltage": 12.0,
    "f1_soil_data": None,
    "nav_running": False,
    "route_status": {}
}

# ===== 前端页面入口 =====
@app.route("/")
def index():
    # 返回 Web 地面站页面
    return render_template("index.html")

# ===== 指令转发接口 =====
@app.route("/api/gcs/cmd<cmd>", methods=["POST"])
def send_cmd_via_post(cmd):
    # 前端点击按钮后, 后端将指令转发给 RK3588
    threading.Thread(target=send_cmd_to_rk, args=(cmd,), daemon=True).start()
    return jsonify({"ok": True})

@app.route("/api/gcs/start-patrol", methods=["POST"])
def start_patrol():
    # 启动全自动巡检
    threading.Thread(target=send_cmd_to_rk, args=("START_PATROL",), daemon=True).start()
    return jsonify({"success": True})

@app.route("/api/gcs/stop-patrol", methods=["POST"])
def stop_patrol():
    # 停止全自动巡检
    threading.Thread(target=send_cmd_to_rk, args=("STOP_PATROL",), daemon=True).start()
    return jsonify({"success": True})

def send_cmd_to_rk(cmd_str):
    """通过 TCP 将控制指令转发给 RK3588"""
    if not cmd_str.endswith("\n") and not cmd_str.startswith("("):
        cmd_str += "\n"
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(0.2)
        s.connect((RK_IP, RK_CMD_PORT))
        s.sendall(cmd_str.encode("utf-8"))
        s.close()
        return True
    except:
        return False

# ===== 实时视频与遥测接收 =====
def master_compound_stream_worker():
    """连接 RK3588, 接收 图像帧 + JSON遥测 的复合数据包"""
    global latest_data
    while True:
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        try:
            client.connect((RK_IP, RK_PORT))
            while True:
                # 1. 读取图像长度
                img_len = int.from_bytes(client.recv(4), "big")
                # 2. 读取图像数据
                img_data = b""
                while len(img_data) < img_len:
                    img_data += client.recv(img_len - len(img_data))
                # 3. 读取遥测 JSON 长度
                txt_len = int.from_bytes(client.recv(4), "big")
                # 4. 读取遥测 JSON 数据
                txt_data = b""
                while len(txt_data) < txt_len:
                    txt_data += client.recv(txt_len - len(txt_data))
                # 5. 解码图像并推送给前端
                img = cv2.imdecode(np.frombuffer(img_data, np.uint8), cv2.IMREAD_COLOR)
                if img is not None:
                    buf = cv2.imencode(".jpg", img)
                    socketio.emit("video_frame", {
                        "image": base64.b64encode(buf).decode("utf-8")
                    })
                # 6. 解析遥测数据并推送给前端
                pack = json.loads(txt_data.decode("utf-8"))
                latest_data["car_world_x"] = pack.get("car_world_x", 0.0)
                latest_data["car_world_y"] = pack.get("car_world_y", 0.0)
                latest_data["car_yaw"] = pack.get("car_yaw", 0.0)
                latest_data["recorded_path"] = pack.get("recorded_path", [])
                latest_data["permanent_obstacles"] = pack.get("permanent_obstacles", [])
                latest_data["semantic_markers"] = pack.get("semantic_markers", [])
                latest_data["battery_voltage"] = pack.get("battery_voltage", 12.0)
                latest_data["f1_soil_data"] = pack.get("f1_soil_data")
                latest_data["nav_running"] = pack.get("nav_running", False)
                latest_data["route_status"] = pack.get("route_status", {})
                socketio.emit("telemetry_update", latest_data)
            except:
                socketio.sleep(0.5)
            finally:
                client.close()

# ===== 任务资产接收与归档 =====
def background_assets_receiver():
    """接收 RK3588 上传的报告、原始视频、AI标注视频并保存"""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```



```
# ===== 任务资产接收与归档 =====
def background_assets_receiver():
    """接收 RK3588 上传的报告、原始视频、AI标注视频并保存"""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(("0.0.0.0", ASSETS_RECV_PORT))
    server.listen(10)
    while True:
        conn, _ = server.accept()
        # 1. 读取文件头: 文件名:文件大小
        header = b""
        while b"\n" not in header:
            header += conn.recv(1)
        file_name, file_size = header.decode("utf-8").strip().split(":")
        file_size = int(file_size)
        # 2. 按任务时间戳建立文件夹
        task_folder = os.path.join(BASE_SAVE_PATH, "Task_" + datetime.now().strftime("%Y%m%d_%H%M%S"))
        os.makedirs(task_folder, exist_ok=True)
        save_path = os.path.join(task_folder, file_name)
        # 3. 分块接收文件并写入本地
        received = 0
        with open(save_path, "wb") as f:
            while received < file_size:
                chunk = conn.recv(min(65536, file_size - received))
                if not chunk:
                    break
                f.write(chunk)
                received += len(chunk)
        conn.close()

# ===== 服务启动 =====
if __name__ == "__main__":
    threading.Thread(target=background_assets_receiver, daemon=True).start()
    socketio.start_background_task(master_compound_stream_worker)
    socketio.run(app, host="0.0.0.0", port=5000, debug=False)
```

第三部分 完成情况及性能参数

3.1 整体介绍（整个系统实物的正面、斜 45°全局性照片）

目前作品已完成履带底盘、RK3588 主控、STM32 执行控制、激光雷达、摄像头、喷药机构、土壤检测机构 and PC 地面站的联调。实物系统能够完成远程手动控制、实时视频显示、激光雷达地图显示、作物与杂草识别、语义点标绘、参数化巡检、喷药触发、土壤数据解析和任务归档。

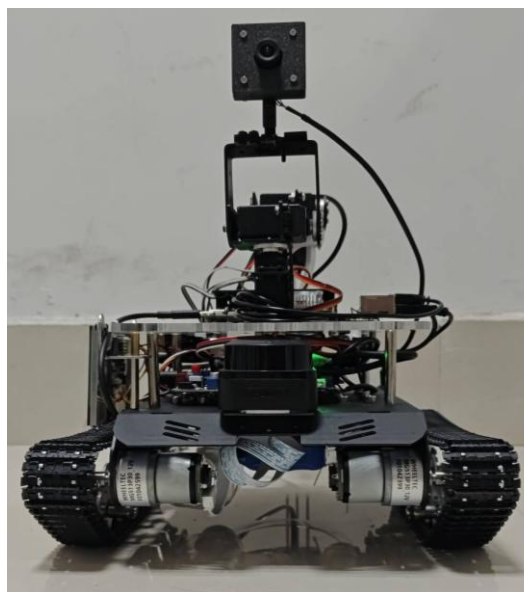


图 3-1 正面实物图

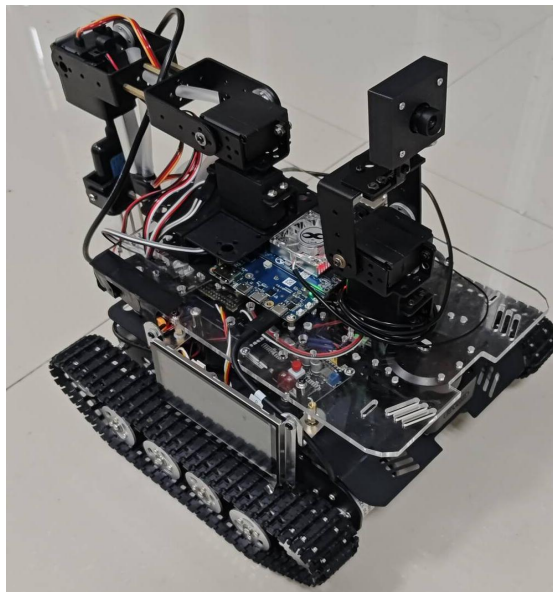


图 3-2 斜 45°全局性照片

3.2 工程成果（分硬件实物、软件界面等设计结果）

3.2.1 机械成果；（实物照片）

机械部分完成了履带移动平台、摄像头与雷达安装、喷药机械臂和土壤采样机构的集成。底盘为传感器和执行机构提供安装空间，喷药机构能够根据主控命令执行不同方向的喷药动作，土壤机构能够配合左右侧检测命令完成定点采样。

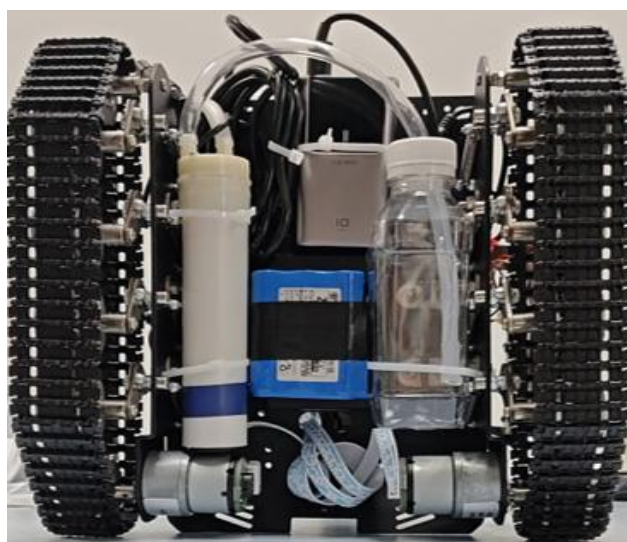


图 3-3 底盘履带移动平台

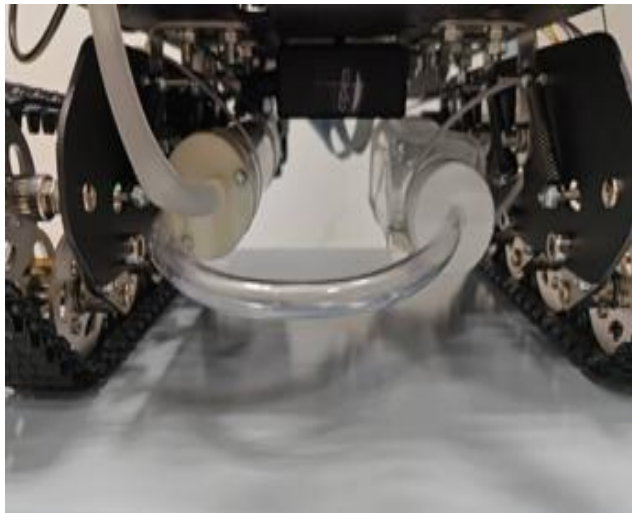


图 3-4 喷药装置

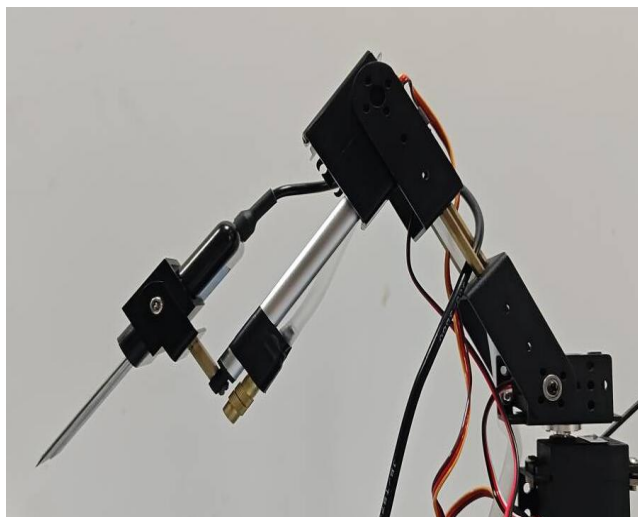


图 3-5 土壤采样和机械臂伸缩装置特写图

3.2.2 电路成果；

电路部分完成了 RK3588、STM32 控制板、传感器接口、执行机构接口和电源系统的连接。串口链路能够稳定承担上层控制指令和底层数据回传任务，ROS2 数据网桥已能解析里程计、电压和土壤肥力数据。

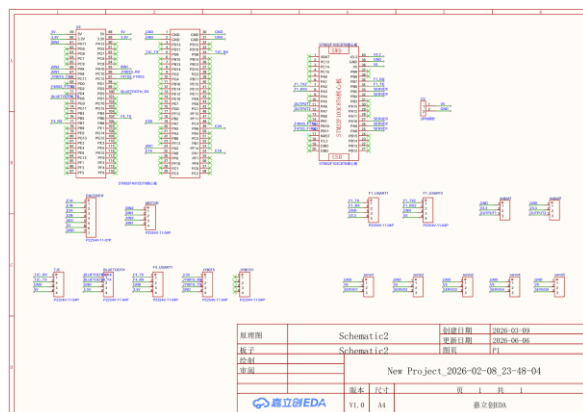


图 3-6 STM32 主控原理图

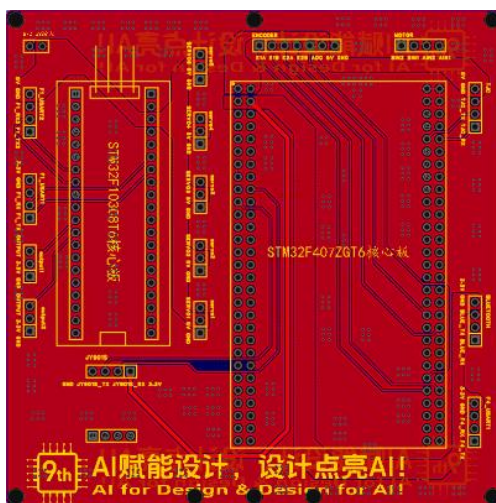


图 3-7 STM32 控制板

3.2.3 软件成果；

软件部分完成了 RK3588 主控程序、ROS2 数据网桥、雷达过滤脚本、Cartographer 建图启动文件、自动巡检状态机、PC 端 Flask 服务器、Web 前端和报告生成脚本。Web 页面可以实时显示视频、坐标、航向角、电池电压、作物计数、地图轨迹和巡航状态，并支持手动控制、喷药开关、土壤检测和地图重置。

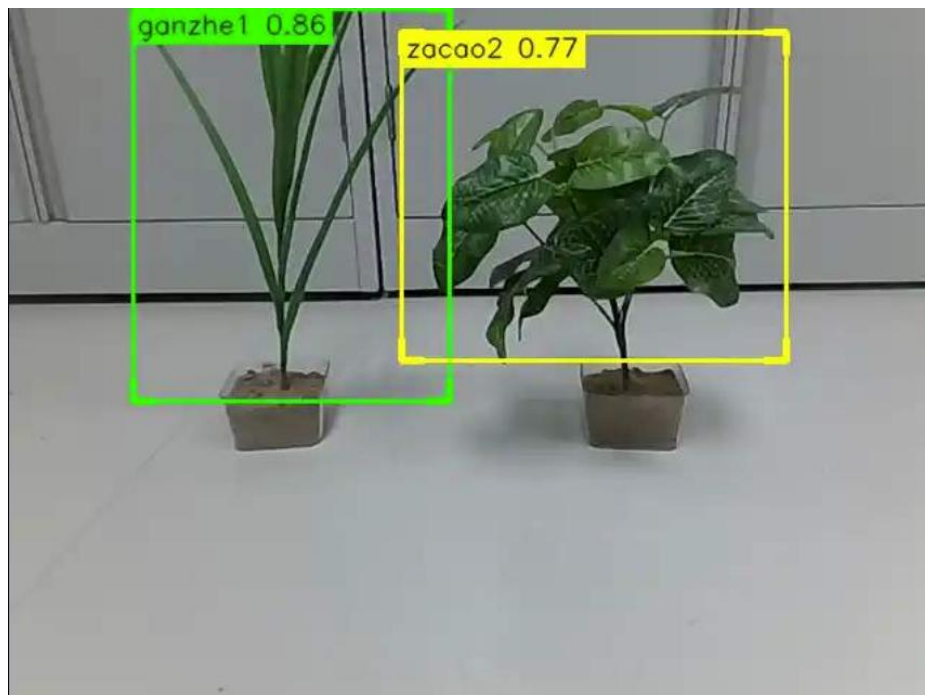


图 3-8 YOLOv8 检测界面

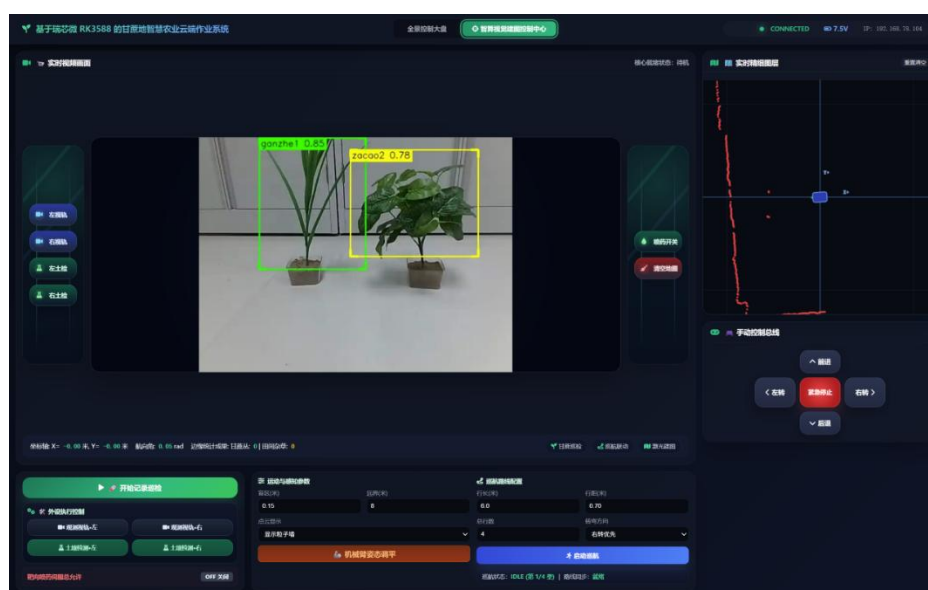


图 3-9 Web 管理界面

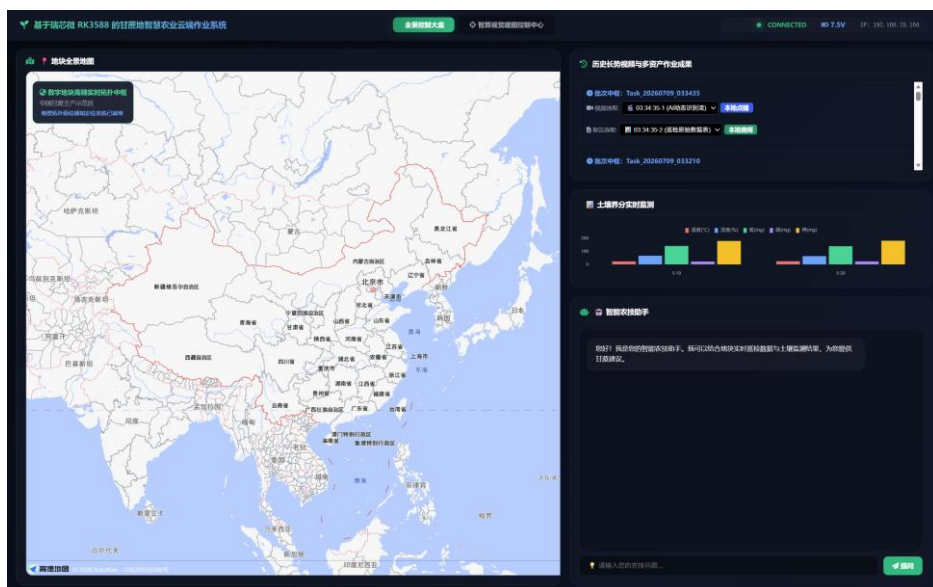


图 3-10 数字化归档界面

3.3 特性成果（逐个展示功能、性能参数等量化指标）（可加重要仪器测试或现场照片）；

成果项目	实现情况	价值
全自动巡检	行长、行距、行数、转向 方向可配置	适配不同垄距和比赛演示 路线
数字化语义地图	轨迹、雷达点云、杂草 点、作物点和土壤点统一 显示	将田间巡检从“看一眼”升 级为“可管理的数据资产”
定向喷药	视觉识别杂草，雷达判断 方位，机械臂按方向喷 药，已喷目标点进入队 列，附近目标跳过	减少无目标喷洒，避免同 一杂草点反复触发便于后 续药量闭环
报告归档	原始报告、专家报告、原 始视频、检测视频自动保 存并同步	便于赛后展示和农户复盘
本地服务器	PC 端接收实时数据与历 史文件	无公网环境下仍可形成端 边协同闭环

第四部分 总结

4.1 可扩展之处

系统后续可从四个方向扩展：

第一在定位与导航方面，可在现有里程计和 Cartographer 建图基础上补充 EKF 融合，将编码器、IMU、激光雷达位姿和航向角进行融合，提高车辆在泥地打滑、垄沟起伏和低速转向情况下的位姿稳定性。后续可扩展为基于垄线识别、局部路径规划和障碍物避让的自主导航，使系统能够适应更复杂的真实蔗田环境。

第二，在视觉识别方面，可继续扩充甘蔗苗期、中期、杂草、病害、缺苗、倒伏等多类别数据集，提升模型对不同光照、遮挡、泥土背景和复杂植被条件的适应能力。后续还可以加入轻量化分割模型或多目标跟踪算法，使系统不仅能判断“是否存在杂草”，还能估计杂草区域面积、密度和连续分布范围，为药量估算和农情评估提供更细粒度依据。

第三，在云端与本地服务器方面，可将 PC 本地服务器迁移到云服务器、对象存储或边缘网关平台，实现多设备、多地块、多任务的集中管理。当前系统已经具备本地视频、报告和历史文件归档能力，后续只需扩展用户权限、地块编号、任务检索、远程下载和移动端访问接口，就可以形成面向农户、合作社或农机服务队的轻量化云端管护平台。

第四，在数字化地图方面，可将当前轨迹、点云、作物点、杂草点和土壤点进一步扩展为多图层农田管理平台。后续可为每个地块建立长期档案，记录不同日期的巡检轨迹、杂草热力分布、土壤肥力变化和喷药记录，形成时间维度上的农情变化曲线。这样系统不仅能完成单次作业展示，还能服务于长期田间管理、复查补喷和农技决策。

第五，在应用场景方面，该系统不仅适用于甘蔗地，也可迁移到玉米、烟草、蔬菜苗期等行作物场景。只需更换视觉识别模型、调整垄距参数和喷药执行策略，即可适配不同作物的巡检、除草、土壤检测和作业归档需求。

4.2 心得体会

本作品研发过程中最大的体会是，农业机器人不能只追求单个算法指标，更要解决“现场数据如何变成可管理资产”的问题。最初设计时容易把重点放在识别模型或底盘运动上，但真正联调后发现，评委和用户更关心的是机器人走过哪里、

发现了什么、哪些目标已经处理、报告能不能保存、历史数据能不能回看。因此，本项目最终把技术主线调整为“视觉+激光雷达构建数字化甘蔗地地图”，这比单纯展示一个识别框更能体现智慧农业价值。

第二个体会是，工程系统需要扬长避短。本系统将自动巡航定位为全自动巡检，将喷药距离和时长定位为标定参数方案，把已经稳定实现的语义地图、目标方位判断、喷药状态机、防重复喷施和本地报告归档作为核心亮点。这样既不夸大，也能充分体现系统的工程完成度和扩展潜力。

第三个体会是，嵌入式系统的价值在于多模块协同。RK3588 提供视觉推理和 ROS2 建图能力，STM32 提供稳定的底层执行，PC 本地服务器提供数据管理和可视化，三者缺一不可。通过统一串口协议、Socket 数据流、HTTP 文件接口和 Web 前端，本作品把硬件、软件和农业场景连接起来，形成了一个可以演示、可以复盘、可以继续升级的完整平台。后续若进一步完善机械臂距离闭环和云端部署，该系统可以从比赛作品逐步演进为真实农田数字化管护装备。

第五部分 参考文献

[1] 姜华骏.基于二维激光雷达的蔗田地貌检测方法研究 [D]. 南宁: 广西大学, 2023.

[2] 张莹, 朱奕庚, 吴林峰, 等. 国内外苗间除草装置研究现状与展望 [J]. 农机使用与维修, 2024 (4): 62-65.

[3] 李宏伟, 赖信东, 姚烨, 等. 幼苗期宿根蔗田机器人自主循垄导航方法 [J]. 农业机械学报, 2026, 57 (9): 127-137.

[4]姜迪,庄卫东. 农业机器人视觉导航技术的应用研究与展望[J].现代化农业,2024, (12):87-90.

[5]陈鹏.基于激光雷达与视觉融合的 SLAM 系统设计与实现[D].南京邮电大学,2021.DOI:10.27251/d.cnki.gnjdc.2021.000417.

[6] 我国农业机器人现状与发展趋势[J].农业工程技术,2020,40 (3):33-35.

[7]沈明霞,姬长英. 农业机器人视觉导航技术发展展望[J].农业机械学报,2001, (1):109-111+117.